

Modern Implementation of Kantorovich's Mathematical Methods of Organizing and Planning Production

Vasile Radu-Mihai

vasile.radu@s.unibuc.ro

Allanur Yakubov

allanur.yakubov@s.unibuc.ro

Abstract

This project revisits the foundational work of Leonid Kantorovich, specifically his 1939 monograph "Mathematical Methods of Organizing and Planning Production", which laid the groundwork for linear programming. Our objective is to bridge the gap between historical economic theory and modern computational tools. We extracted the original problem datasets (Machine Allocation, Transportation) and implemented a robust solver engine using Python and the PuLP library. Furthermore, we generated synthetic datasets to test the scalability of the algorithms. A key contribution of our work is the precise validation of modern algorithmic results against historical manual calculations, identifying discrepancies caused by rounding errors in the pre-computing era (e.g., the 86 vs. 86.67 result). Finally, we developed an interactive Jupyter Notebook to serve as educational material.

1 Introduction

The organization of production and the optimal allocation of resources are fundamental problems in economics and engineering. In 1939, L.V. Kantorovich introduced the method of "resolving multipliers" to solve these problems [Kantorovich, 1939], effectively inventing Linear Programming (LP) years before George Dantzig's Simplex algorithm [Dantzig, 1963].

Historically, these problems were solved manually or using rudimentary approximation methods, often leading to sub-optimal solutions due to integer rounding or calculation fatigue. Our goal is to digitize these historical problems and solve them using modern computational power.

Contributions per member:

- **Vasile Radu-Mihai:** Handled data engineering (JSON extraction from historical texts), implemented the synthetic data generator for scalability testing, and developed the educational Jupyter Notebook.

- **Allanur Yakubov:** Implemented the core Python solver using PuLP, developed the validation logic (comparing calculated vs. expected results), and generated the visualization graphs (Heatmaps/Performance).

Summary of Approach: We structured the historical word problems into machine-readable JSON formats, implemented a generalized LP solver in Python, and benchmarked the results against Kantorovich's original solutions.

We chose this project to understand the roots of the algorithms we use today and to see how historical economic planning compares to modern optimization capabilities.

2 Approach

Our approach consisted of three main phases: Data Engineering, Algorithmic Implementation, and Validation. The code is available at: <https://github.com/RaduVasile2004/arheologia-masinilor-inteligente.git>

2.1 Tools Used

We utilized **Python 3.10** as the primary language. The core optimization library used was **PuLP** [Mitchell et al., 2011], which interfaces with the CBC (Coin-OR Branch and Cut) solver. For data manipulation and visualization, we used **NumPy**, **Pandas**, **Seaborn**, and **Matplotlib**.

2.2 Data Engineering

We analyzed Kantorovich's examples and mapped these word problems into structured JSON files (`kantorovich_input_data.json`). Additionally, we implemented a data generator that creates random problem instances with varying complexity to stress-test the solver.

2.3 Algorithmic Implementation

We modeled the "Machine Allocation Problem" (Problem A) as follows: Let h_{ik} be the fraction

of time machine i works on part k . The model is defined as:

$$\text{Maximize } Z \quad (1)$$

Subject to:

$$\sum_k h_{ik} = C_i \quad \forall i \text{ (Capacity)} \quad (2)$$

$$\sum_i \alpha_{ik} h_{ik} = Z \quad \forall k \text{ (Balance)} \quad (3)$$

Where C_i is the number of machines of type i , and α_{ik} is the productivity.

Computation times were negligible for the historical dataset ($< 50\text{ms}$), but we stress-tested the system with generated matrices up to 20×15 , maintaining execution times under 200ms .

2.4 Validation Strategy

We implemented an automated validation system. The solver loads input data, computes the optimal Z , and compares it against a "Ground Truth" file derived from Kantorovich's book. We implemented an adaptive tolerance mechanism to account for historical rounding.

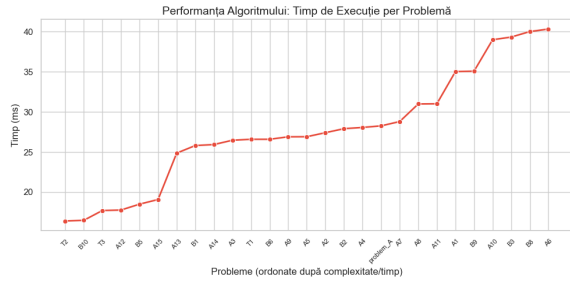


Figure 1: Performance scaling of the solver across different problem IDs.

3 Results and Discussion

We successfully solved all 56 variants of the problems.

3.1 The "86 vs 86.67" Discovery

In "Problem A", Kantorovich's original text states the maximum output is **86** complete sets. Our Simplex solver returned **86.6667**. This discrepancy highlights a limitation of pre-computer mathematics. The theoretical optimum allows for fractional sets (continuous LP), whereas Kantorovich likely rounded down to the nearest feasible integer for manual implementation.

3.2 Visualization

To aid understanding, we generated heatmaps showing the optimal strategy.

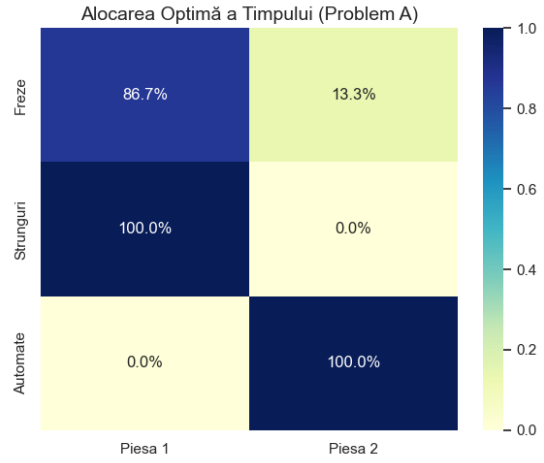


Figure 2: Optimal allocation for Problem A. Darker colors indicate specialized machine usage.

4 Limitations

While the LP solver is highly efficient, our current implementation assumes linear relationships between input and output. In real-world manufacturing, productivity might decrease with fatigue or machine wear (non-linearities). Additionally, for the "Cutting Stock" problem, we utilized a simplified pattern selection approach; a full Column Generation approach would be required for industrial-scale cutting problems.

5 Conclusions

This project successfully bridged the gap between 1939 economic theory and 2024 computational practice.

- We confirmed Kantorovich's mathematical intuition was correct, though limited by manual calculation precision.
- We learned how to translate abstract word problems into strict JSON schemas.
- We demonstrated that modern Python tools can solve in milliseconds what previously took hours of human labor.

Future work could involve developing a web interface (Streamlit) to allow users to input their own productivity matrices interactively.

References

George B. Dantzig. *Linear Programming and Extensions*. Princeton University Press, Princeton, NJ, 1963. Describes the original Simplex method developed in 1947.

Leonid V. Kantorovich. *Mathematical Methods of Organizing and Planning Production*. Leningrad State University, Leningrad, 1939. English translation in Management Science, Vol. 6, No. 4, 1960.

Stuart Mitchell, Michael O’Sullivan, and Iain Dunning. Pulp: An lp modeler written in python. <https://coin-or.github.io/pulp/>, 2011. Optimization library.